Artificial Intelligence

AI-2002

Lab 07

Constraint Satisfaction Problems (CSP)

National University of Computer & Emerging Sciences – NUCES – Karachi



Course Code: AI-2002	Artificial Intelligence Lab

1. Objective 3
2. Introduction to CSP 3
3. Assignments and Domains in CSP: 3
1. **Consistent (Legal) Assignment:** 3
2. **Complete Assignment:** 3
3. **Partial Assignment:** 4
4. Domains: 4
5. Code Examples 5
6. LAB_TASK: 16

1. Objective

1. Learn how to represent real-world problems as variables, domains, and constraints.
2. Gain knowledge of backtracking search, constraint propagation, and heuristics to solve CSPs efficiently.

2. Introduction to CSP

To satisfy the constraint according to the set of rules is known as constraint satisfaction problem. CSP plays an important part in Artificial Intelligence with csp's all the decision making problems solved easily when its values meet specific constraints or rules of the problem.

The CSP is defined through the three variables **X (**set of variables**) , D (**set of domains**) where** each Variables has specific domains and **C (**set of constraints**)** where f(C)->X
Constraint values can be represented as a **pair{scope, rel}** where:
Scope is a tuple of variables present in a constraint and **Relation** is a combination or set of allowed values of the variables.

3. Assignments and Domains in CSP:

1. Consistent (Legal) Assignment :

An assignment is consistent if it does not violate any constraints of the problem. In other words, all the assigned values satisfy the given constraints.
For example: Map Coloring Problem in the game of LUDO all players can take unique color in consistent (Legal) Assignment suppose four players are playing this game Player_1 = Red , Player_2 = Yellow , Player_3 = Green , Player_4 = Blue so the variables , domains and constraints should be:

- **Variables:** {Player_1, Player_2, Player_3, Player_4}
- **Domains:** {Red, Blue, Green, Yellow}
- **Constraints:** Player_1 ≠ Player_2, Player_2 ≠ Player_3, Player_1 ≠ Player_3

2. Complete Assignment:

A complete assignment should be where each and every variable assigns a value and the solution is to satisfy the constraints.
For example: Sudoku Game in which a complete assignment assigns each row and column a unique value containing values to all variables.

1	4	3	2
2	3	4	1

- **Variables:** Each cell in the grid.
- **Domains:** Possible values {1,2,3,4} (or {1-9} for a 9x9 Sudoku).
- **Constraints:** Uniqueness in rows, columns, and sub-grids.
- **Goal:** Find a **complete and valid assignment** satisfying all constraints.

3. Partial Assignment:

Those assignments who assign values to some variables are known as partial assignments.

For example: In Sudoku Game Partial Assignment assigns values to some variables not all.

_	1	___	3	___
2	___	4	___	3

4. Domains:

Domains are used by the variables

- **Discrete Domain:**
  This domain is infinite where multiple variables can be mapped to one state For Example: Multiple Variables can use start state infinite times.
- **Finite Domain:**
  A finite domain containing one domain for one variable can be a specific variable in a form of continuous states.

Constraints in CSP:

1. **Unary Constraints:**
   It is the simplest type of constraint that restricts the value of a single variable.
2. **Binary Constraints:**
   Binary means “two” variables lies in this constraint where the value of one variable contains the set of variables values that shows the relationship between two variables for example:if x2 is constrained by x1 and x3, it means that the possible values of x2 are influenced by or lie within a set defined by x1and x3.
3. **Global Constraints:**
   The variables are arbitrary in this constraint and have no relationship or connection or pattern between them.
4. **Linear Constraints:**
   The variables in this constraint contain integer values in linear form.
5. **Non-linear Constraints:**
   The variables in this constraint contain integer values in a non-linear form.
6. **Constraint propagation in CSP:**
   It is a special type of inference that helps in reducing the legal number of values for the variables and makes the propagation local consistent in the form of nodes. We solve the constraint satisfaction problem with this inference.
7. **Node Consistency:**
   A variable is considered node consistent if every value in its domain satisfies the unary constraints imposed on it.
8. **Arc Consistency:**
   A variable is arc consistent if each value in its domain satisfies the binary constraints with respect to the related variables.
9. **Path Consistency:**
   When the evaluation of a pair of variables concerning a third variable can be extended to another variable while satisfying all binary constraints, it is similar to arc consistency.
10. **k-consistency:**
    This type of consistency is used to establish stronger forms of propagation.

5. Code Examples

```
!pip install ortools

Collecting ortools
  Downloading ortools-9.11.4210-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (3.0 kB)
Collecting absl-py>=2.0.0 (from ortools)
  Downloading absl_py-2.1.0-py3-none-any.whl.metadata (2.3 kB)
Requirement already satisfied: numpy>=1.13.3 in /usr/local/lib/python3.11/dist-packages (from ortools) (1.26.4)
Requirement already satisfied: pandas>=2.0.0 in /usr/local/lib/python3.11/dist-packages (from ortools) (2.2.2)
Collecting protobuf<5.27,>=5.26.1 (from ortools)
  Downloading protobuf-5.26.1-cp37-abi3-manylinux2014_x86_64.whl.metadata (592 bytes)
Requirement already satisfied: immutabledict>=3.0.0 in /usr/local/lib/python3.11/dist-packages (from ortools) (4.2.1)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.11/dist-packages (from pandas>=2.0.0->ortools) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-packages (from pandas>=2.0.0->ortools) (2024.2)
```

Example # 1

Define a CSP problem with three variables ('a', 'b', 'c') that can take on values between 1 and 3. We add constraints to the problem to ensure that no two variables can have the same value. We then call the getSolutions method to find all possible solutions to the problem

Calling CPMODEL top create the CSP Model using ortools

```
from ortools.sat.python import cp_model
# Create the model
model = cp_model.CpModel()
```

Adding Variables

```
# Define the variables with a domain of {1, 2, 3}
a = model.NewIntVar(1, 3, 'a') #To add variable we use .NewIntVar
Method
b = model.NewIntVar(1, 3, 'b')
c = model.NewIntVar(1, 3, 'c')
```

Defining Constraint

```
# Add constraints to ensure all variables have different values
model.Add(a != b) #To add constraints we use .Add Method
model.Add(b != c)
model.Add(a != c)
```

Calling Solver to Solve the CSP Problem

```
# Create the solver
solver = cp_model.CpSolver()
# VarArray Solution for generating all possible solution following
the constraints
solution_printer = cp_model.VarArraySolutionPrinter([a, b, c])
# Setting enumerate to True , to display all records
solver.parameters.enumerate_all_solutions = True
```

Getting all Solutions

```
# Solve the problem
solver.Solve(model, solution_printer)
print("optimal",cp_model.OPTIMAL)
print("Feasible",cp_model.FEASIBLE)
print("InFeasible",cp_model.INFEASIBLE)
print("UNKNOWN",cp_model.UNKNOWN)
```

Example # 2

A manager is assigning shifts to three employees: Alice, Bob, and Charlie. Each employee must be assigned a shift, and there are three available shifts (0, 1, or 2).

However, there is one condition: Alice and Bob cannot be assigned the same shift (to ensure proper coverage).

- **Variables:** {X, Y, Z}
- **Domains:** {0, 1, 2}
- **Constraint:** X ≠ Y

```
#Import the google ortools library to solve the csp problem
from ortools.sat.python import cp_model

# Declare the model and bind it with CpModel which is already
present in ortools library
model = cp_model.CpModel()

#Declaring the set of variables for csp
num_vals = 3
x = model.new_int_var(0, num_vals - 1, "x")
y = model.new_int_var(0, num_vals - 1, "y")
z = model.new_int_var(0, num_vals - 1, "z")

#Declaring Constraints
model.add(x != y)

# # Create the solver
solver = cp_model.CpSolver()
solution_printer = cp_model.VarArraySolutionPrinter([x, y, z])
solver.parameters.enumerate_all_solutions = True

# Solve the problem
solver.Solve(model, solution_printer)
```

solver.Solve(model, solution_printer) will return the solution

Output

```
Solution 0, time = 0.00 s
  x = 1   y = 2   z = 0
Solution 1, time = 0.01 s
  x = 0   y = 2   z = 0
Solution 2, time = 0.01 s
  x = 0   y = 1   z = 0
Solution 3, time = 0.01 s
  x = 1   y = 0   z = 0
```

```
Solution 4, time = 0.01 s
  x = 1   y = 0   z = 1
Solution 5, time = 0.01 s
  x = 1   y = 2   z = 1
Solution 6, time = 0.01 s
  x = 0   y = 2   z = 1
Solution 7, time = 0.02 s
  x = 0   y = 1   z = 1
Solution 8, time = 0.02 s
  x = 0   y = 1   z = 2
Solution 9, time = 0.02 s
  x = 0   y = 2   z = 2
Solution 10, time = 0.02 s
  x = 1   y = 2   z = 2
Solution 11, time = 0.02 s
  x = 1   y = 0   z = 2
Solution 12, time = 0.03 s
  x = 2   y = 0   z = 2
Solution 13, time = 0.03 s
  x = 2   y = 0   z = 1
Solution 14, time = 0.03 s
  x = 2   y = 0   z = 0
Solution 15, time = 0.03 s
  x = 2   y = 1   z = 0
Solution 16, time = 0.03 s
  x = 2   y = 1   z = 1
Solution 17, time = 0.03 s
  x = 2   y = 1   z = 2
```

In order to achieve the feasible / Optimal /Unknown/ Infeasible solutions, use these methods.

- **cp_model.OPTIMAL**: A solution is found, and it is the best possible solution.
- **cp_model.FEASIBLE**: A valid solution is found, but it may not be optimal.
- **cp_model.INFEASIBLE**: No solution satisfies all constraints.
- **cp_model.UNKNOWN**: The solver could not determine a solution.

Here we are extracting Optimal / Feasible solution if Exist

```
# Solve the model
status = solver.Solve(model)
print(status)
# Display solver status in a readable way
status_mapping = {
    cp_model.OPTIMAL: "OPTIMAL",
    cp_model.FEASIBLE: "FEASIBLE",
    cp_model.INFEASIBLE: "INFEASIBLE",
    cp_model.UNKNOWN: "UNKNOWN"
}
```

```
print(f"Solver Status: {status_mapping.get(status, 'UNKNOWN')}
({status})")

# Print solution if feasible or optimal
if status == cp_model.OPTIMAL or status == cp_model.FEASIBLE:
    print(f"x = {solver.Value(x)}")
    print(f"y = {solver.Value(y)}")
    print(f"z = {solver.Value(z)}")
else:
    print("No feasible solution found.")
```

EXAMPLE # 3

A robotic drone operates in a grid-based environment where it can only move diagonally due to terrain constraints. Given the drone's starting position (Point A) and its destination (Point B), the AI system must determine the shortest direct path using Pythagoras' theorem.

```
import heapq
import math

class DronePathPlanner:
    def __init__(self, grid, start, goal):
        self.grid = grid  # 2D Grid Map (1 = Open, 0 = Obstacle)
        self.start = start  # (x, y) Starting Position
        self.goal = goal  # (x, y) Destination
        self.rows = len(grid)
        self.cols = len(grid[0])
        self.directions = [
    (-1,0),(1,0),(0,-1),(0,1),   # straight moves
    (-1,-1),(-1,1),(1,-1),(1,1)  # diagonal moves
] # Diagonal moves
        self.cost = math.sqrt(2)  # Diagonal movement cost using
Pythagoras' theorem

    def is_valid(self, x, y):
        """Check if the cell is within bounds and not an obstacle."""
        return 0 <= x < self.rows and 0 <= y < self.cols and
self.grid[x][y] == 1

    def ucs(self):
        """Uniform Cost Search to find the shortest diagonal path."""
        priority_queue = [(0, self.start)]  # (Cost, (x, y))
        visited = set()
        parent = {self.start: None}  # Track path
```

```
        while priority_queue:
            cost, (x, y) = heapq.heappop(priority_queue)

            if (x, y) in visited:
                continue

            visited.add((x, y))

            if (x, y) == self.goal:
                return self.reconstruct_path(parent)

            for dx, dy in self.directions:
                new_x, new_y = x + dx, y + dy

                if self.is_valid(new_x, new_y) and (new_x, new_y) not
in visited:
                    heapq.heappush(priority_queue, (cost + self.cost,
(new_x, new_y)))
                    parent[(new_x, new_y)] = (x, y)  # Store path

        return None  # No path found

    def reconstruct_path(self, parent):
        """Reconstruct path from goal to start."""
        path = []
        node = self.goal
        while node:
            path.append(node)
            node = parent[node]
        return path[::-1]  # Reverse to get start-to-goal order

# Define a 5x5 grid (1 = Open, 0 = Obstacle)
grid = [
    [1, 1, 1, 1, 1],
    [1, 0, 1, 0, 1],
    [1, 1, 1, 1, 1],
    [1, 0, 1, 0, 1],
    [1, 1, 1, 1, 1]
]

start = (0, 0)  # Top-left corner
goal = (4, 4)  # Bottom-right corner

# Run UCS for path planning
drone = DronePathPlanner(grid, start, goal)
shortest_path = drone.ucs()
```

```
# Print result
if shortest_path:
    print("Shortest Path:", shortest_path)
else:
    print("No valid path found!")
```

EXAMPLE # 4 (N Queen Problem)

In a security system, an AI must place **N drones on an N × N grid** for surveillance. Each drone can monitor **horizontally, vertically, and diagonally**, similar to a chess queen. To avoid signal interference, the AI must place the drones so that **no two drones are in the same row, column, or diagonal**. This placement problem is modeled as a **Constraint Satisfaction Problem (CSP)** where the goal is to find a valid configuration for all drones without conflicts.

```
import sys
import time
from ortools.sat.python import cp_model
model = cp_model.CpModel()
```

```
# There are `board_size` number of variables, one for a queen in each column
# of the board. The value of each variable is the row that the queen is in.
board_size = 8
queens = [model.new_int_var(0, board_size - 1, f"x_{i}") for i in range(board_size)]

# All rows must be different.
model.add_all_different(queens)

# No two queens can be on the same diagonal.
model.add_all_different(queens[i] + i for i in range(board_size))
model.add_all_different(queens[i] - i for i in range(board_size))

# Add constraints to prevent queens from attacking diagonally.
diag1 = []
for i in range(board_size):  # Reintroduce the loop to define 'i'
    q1 = model.NewIntVar(0, 2 * board_size, 'diag1_%i' % i)
    diag1.append(q1)
    # Need to define j, assuming you want it to be the same as i:
    model.Add(q1 == queens[i] + i)

model.AddAllDifferent(diag1)
```

```
class NQueenSolutionPrinter(cp_model.CpSolverSolutionCallback):
    """Print intermediate solutions."""

    def __init__(self, queens: list[cp_model.IntVar]):
        cp_model.CpSolverSolutionCallback.__init__(self)
        self.__queens = queens
        self.__solution_count = 0
        self.__start_time = time.time()

    @property
    def solution_count(self) -> int:
        return self.__solution_count

    def on_solution_callback(self):
        current_time = time.time()
        print(
            f"Solution {self.__solution_count}, "
            f"time = {current_time - self.__start_time} s"
        )
        self.__solution_count += 1

        all_queens = range(len(self.__queens))
        for i in all_queens:
            for j in all_queens:
                if self.value(self.__queens[j]) == i:
                    # There is a queen in column j, row i.
                    print("Q", end=" ")
                else:
                    print("_", end=" ")
            print()
        print()
```

```
solver = cp_model.CpSolver()
solution_printer = NQueenSolutionPrinter(queens)
solver.parameters.enumerate_all_solutions = True
solver.solve(model, solution_printer)
```

OUTPUT

```
Solution 0, time = 0.012843847274780273 s
_ _ _ _ Q _ _ _
_ _ _ _ _ _ Q _
_ Q _ _ _ _ _ _
_ _ _ _ _ Q _ _
_ _ Q _ _ _ _ _
Q _ _ _ _ _ _ _
```

```
_ _ _ _ _ _ _ Q
_ _ _ Q _ _ _ _
```

..... Solution 1,2,3,....

.

```
Solution 91, time = 0.09277892112731934 s
_ _ _ _ Q _ _ _
Q _ _ _ _ _ _ _
_ _ _ Q _ _ _ _
_ _ _ _ _ Q _ _
_ _ _ _ _ _ _ Q
_ Q _ _ _ _ _ _
_ _ _ _ _ _ Q _
_ _ Q _ _ _ _ _

<CpSolverStatus.OPTIMAL: 4>
```

Other Variable Creation Methods

- **Integer Variables (NewIntVar)** → Represents a variable that can take values within a specific integer range (e.g., x ∈ [0, 10]).
- **Boolean Variables (NewBoolVar)** → A special case of integer variables that can only be 0 or 1, used for binary decisions.
- **Interval Variables (NewFixedSizeIntervalVar)** → Defines a fixed-length time interval (e.g., a task with a known duration).
- **Optional Interval Variables (NewOptionalFixedSizeIntervalVar)**→ Similar to interval variables but only active if a Boolean variable is 1 (e.g., conditional scheduling tasks).

```python
from ortools.sat.python import cp_model

# Create model
model = cp_model.CpModel()

# 1. Creating Integer Variables
x = model.NewIntVar(0, 10, 'x')  # x ∈ [0, 10]
y = model.NewIntVar(-5, 5, 'y')  # y ∈ [-5, 5]

# 2. Creating Boolean Variables (Binary 0/1)
b1 = model.NewBoolVar('b1')  # Boolean variable (0 or 1)
b2 = model.NewBoolVar('b2')

# 3. Creating Interval Variables (for scheduling problems)
start = model.NewIntVar(0, 10, 'start')  # Start time
size = 5  # Fixed duration
```

```
interval = model.NewFixedSizeIntervalVar(start, size,
'interval_task')

# 4. Creating Optional Interval Variable (only valid if `b1` is 1)
optional_interval = model.NewOptionalFixedSizeIntervalVar(start,
size, b1, 'optional_task')

# Print created variables
print(f"Integer Variables: {x}, {y}")
print(f"Boolean Variables: {b1}, {b2}")
print(f"Interval Variable: {interval}")
print(f"Optional Interval Variable: {optional_interval}")
```

Other Constraint Creation Methods

2. Constraint Methods

These methods **add constraints** to the model:

Method	Description
`Add(expr)`	Adds a basic constraint (e.g., `model.Add(x + y == 5)`).
`AddAllDifferent(vars)`	Ensures all variables in `vars` take **unique** values (e.g., sudoku).
`AddElement(index, array, target)`	Ensures `array[index] == target` (useful for lookup tables).
`AddCircuit(arcs)`	Creates a Hamiltonian circuit constraint for routing problems.
`AddAllowedAssignments(vars, tuples)`	Allows only specific assignments for given variables.
`AddForbiddenAssignments(vars, tuples)`	Forbids specific assignments for given variables.
`AddMinEquality(target, vars)`	Forces `target = min(vars)`.
`AddMaxEquality(target, vars)`	Forces `target = max(vars)`.
`AddDivisionEquality(target, num, denom)`	Ensures `target = num // denom`.
`AddMultiplicationEquality(target, factors)`	Ensures `target = factors[0] * factors[1] * ...`.

```
from ortools.sat.python import cp_model

# Create model
model = cp_model.CpModel()

# Define variables
x = model.NewIntVar(0, 10, "x")
y = model.NewIntVar(0, 10, "y")
z = model.NewIntVar(0, 10, "z")

# Applying constraints
model.Add(x + y <= 10)
model.AddAllDifferent([x, y, z])
```

```
# Allowed combinations for x,y
model.AddAllowedAssignments([x, y], [(1, 2), (3, 4)])

# Forbidden combinations
model.AddForbiddenAssignments([x, y], [(1, 1), (2, 2)])

# Min and Max targets (separate variables)
min_target = model.NewIntVar(0, 10, "min_target")
max_target = model.NewIntVar(0, 10, "max_target")

model.AddMinEquality(min_target, [x, y, z])
model.AddMaxEquality(max_target, [x, y, z])

# Division constraint
num = model.NewIntVar(1, 10, "num")
denom = model.NewIntVar(1, 5, "denom")
div_target = model.NewIntVar(0, 10, "div_target")
model.AddDivisionEquality(div_target, num, denom)

# Multiplication constraint
factors = [model.NewIntVar(1, 5, f"f{i}") for i in range(2)]
prod_target = model.NewIntVar(1, 25, "prod_target")
model.AddMultiplicationEquality(prod_target, factors)

# Solver
solver = cp_model.CpSolver()
status = solver.Solve(model)

# Output results if feasible
if status == cp_model.OPTIMAL or status == cp_model.FEASIBLE:

    print("Solution Found\n")

    print("x =", solver.Value(x))
    print("y =", solver.Value(y))
    print("z =", solver.Value(z))

    print("\nMinimum value =", solver.Value(min_target))
    print("Maximum value =", solver.Value(max_target))

    print("\nDivision result =", solver.Value(div_target))
    print("Numerator =", solver.Value(num))
    print("Denominator =", solver.Value(denom))
```

```
    print("\nProduct result =", solver.Value(prod_target))
    print("Factors =", [solver.Value(f) for f in factors])

else:
    print("No feasible solution found.")
```

Output

```
Solution Found

x = 1
y = 2
z = 0

Minimum value = 0
Maximum value = 2

Division result = 1
Numerator = 1
Denominator = 1

Product result = 1
Factors = [1, 1]
```

Define the CSP problem